

Theseus's killer features

2026-10-09 · Tabitha/Claude · one page, a short section per feature, each linking its longer plan. Numbers come from the [benchmarks omnibus](#), the [north-star plan](#) and the reports they cite.

The answer first

A developer asked the owner: "what's Theseus's killer feature?" The honest answer is a set, and the goal behind it is the north star: **match or beat Claude Code at coding and be as pleasant to use, then crush it on memory, personality, speed, scale and task management at every horizon.**

Feature	The pitch a developer would repeat	Where it stands	Plan
Speed	"It's 36 MiB and starts in milliseconds; the others are hundreds of MiB."	measured, ahead on 6 of 9 rows a person feels	north star §1.5
Self-improvement	"It builds its own tools, exams and fixes, joins what passes every gate, and tells me after."	lower rungs live , the rest built now and off	RSI plan
Parallel tool calls	"Four slow test suites in one response finish in the time of one."	designed , in the stopping point	the asyncbench
Task management at every horizon	"Remind me, check back, run this every Friday, tell me when the deploy is green: it all survives a restart."	durable ; ● 3 of 11 horizon tests	north star §1.6
Memory	"It remembers everything I've done with it, and with my old agent, and links the message it came from."	live on the owner's daemon , never measured head to head	north star §1.3
Personality	"It knows who it is and who I am, on every surface, after every compaction."	behind , the fix ready	north star §1.4
Remote runtimes	"The daemon on my server, the CLI on my laptop, the index on another box, over mTLS."	designed , decided	remote runtimes
Speculative activity	"While I read its answer it already fixed the next red test in a sandbox; one key to use it."	designed , decided	speculation
Discord as a live surface	"A lively chat with its thinking, a task house in my guild, and it speaks first when something happens."	ahead (Claude Code has no Discord); half built	north star §1.7

Where Theseus does **not** win yet, said plainly: as a coding agent it solved 71.9% of Terminal-Bench 2.0 trials against Claude Code's 81.5% on the same model (October 4), and on the first-ten-minutes checks it passes 8 of 42. Most of the coding gap was harness faults, four of whose five kinds are fixed; the rerun after the stopping point (theseus-lc7a) says where it stands now. Every claim below that is not yet measured says so.

1. Speed: FAST is a primary goal

The pitch. Theseus is a Rust daemon, not a process per terminal: it starts in milliseconds, holds tens of MiB, and spends a few milliseconds of its own CPU on a turn. Every change must pass the gate's lifecycle budgets, so it stays that way.

The numbers.

- **Against seven other harnesses** (October 7, [omnibus](#)): about 36 MiB of peak memory against 125 to 1,380 MiB; about 17 ms of CPU a tool call against 140 to 4,923 ms; installed and started in a task container in 0.83 s against 19.5 to 304 s.
- **Against Claude Code on the same stand-in model** ([north star §1.5](#)): a ready prompt in 4.8 to 31 ms against 408 to 502 ms (about 50x); the model's first byte on screen in 0.6 to 1.5 ms against 112 to 216 ms (about 100x); a long session resumed in 24 to 31 ms against 583 to 1,647 ms; about 10 ms of CPU a plain turn against 270 to 300 ms; 37 to 40 MB against 249 to 268 MB.
- **In the gate:** cold start 22.6 ms at the median, clean stop 35.1 ms, crash restart 35.6 ms, binary swap 51.7 ms, over 240 gates; 21.5 ms cold at 10,000 sessions. On the owner's real store (21,779 imported sessions, 115,697 nodes) the daemon served in 88.1 ms.
- **Every run made it faster:** the omnibus counts fourteen measured speed-ups, from 1.2 to 2,700 times, each from a finding a benchmark run made.

Where it loses, and the fix. A tool call takes 34 ms for a file read against Claude Code's 23 ms, and the owner's daemon waits 278 ms before a model call. Both are waits, not work: 85% to 90% of the harness's own time on a turn is `fdatsync`, and with the store on tmpfs the same call takes 4.6 ms, 5x faster than Claude Code. Durability stays (a crash loses nothing); the fix is a budget for syncs on the critical path. At 10x load, seven scale cliffs are named (1,017 connections stop the daemon; a 10,000-node session's first turn takes 30.8 s), each with its row.

Plan: [north star §1.5](#) and its rows.

2. Self-improvement: it builds its own hands, and acts first

The pitch. Claude Code is a tool a person configures. Theseus is a daemon that keeps a ledger of everything it did and closes loops on itself: the owner's labels rewrite its judge's wording nightly, corrections re-steer its model routing, its memory consolidates itself, and it can write, trial and (with the owner's Load) use its own tools.

The ladder. Seven rungs: knowledge, judgement, tools and compute hands live or built inside Theseus today; its own code, its own yardsticks and choosing what to improve next are done *for* it by an outer loop today, and the plan moves them inward.

Decided: act first, tell the owner after. For its own code, yardsticks and backlog, Theseus files and ranks its own rows, builds a branch, gets an independent review from a different session, joins when every gate passes, installs, and then tells the owner what changed, why and what it cost. A regression reverts on its own; the owner's veto is one click. **A short hard keel still needs the owner's yes:** deleting or

loosening tests, budgets or ceilings; sealed holdout sets kept out of its reach; one-way store format bumps; security, policy, secret and permission changes; spend limits (\$10 a self-branch, \$30 a day); and a kill switch only the owner turns back on.

Status. The machinery is being built now and ships off; the switch goes on the day the benchmark rerun reports, so the rerun measures Theseus before it started changing itself.

Plan: [the self-improvement \(RSI\) plan](#) (PDF).

3. Parallel tool calls and the asyncbench

The pitch. When the model asks for four slow test suites in one response, they run together and finish in the time of the slowest, and a benchmark proves it.

Today. Reads in one response already run together. Each Write or Run call is a barrier and runs alone, so four `proc.run`s take four times as long as one. Theseus's model already puts more than one call in 21% of its responses on Terminal-Bench (Claude Code's: about 10%), and in three of four write-first responses a later call uses what the write wrote, so "run everything at once" would break real work.

The design (theseus-d1hi, da46, 2wxa, in the stopping point).

- **The rule:** reads with reads, programs with programs, writes with writes when their paths differ; a class change stays a barrier, so write-then-run and send-then-read keep their order. Up to 8 jobs at once by default; results go back in call order, in one message, as today.
- **Minimal prompting, measured.** The owner's order of levers: the API's own contract, the tools' shape, their descriptions and their result metadata first; one factual sentence in the tools paragraph only because a benchmark arm shows whether it earns its place.
- **The asyncbench** extends the existing async bench in two layers. Layer 1 (\$0, the stand-in model): one response of N slow calls, wall time against N, for Theseus before and after, Claude Code, Pi and OpenCode. Layer 2 (a real model, about \$10 to \$25): 20 developer chores whose slow parts are independent (test suites, builds, logs, fetches), a quarter of them controls where batching dependent steps costs correctness, across six arms.
- **Honest about the size of it:** on Terminal-Bench work the change is worth about 0.2% of tool time, and the report will say so. The gain lives in work with independent slow parts.

Plan: theseus-d1hi, theseus-da46 and theseus-2wxa under the stopping point; the report will land in `docs/benchmarks/`.

4. Task management at every horizon

The pitch. "Remind me at 5", "check back on this tomorrow", "do this Friday", "every morning, the status", "tell me when the deploy is green", "the launch in three weeks": one model for all of it, durable across restarts and installs, visible on every surface.

Today. The timer underneath is durable: wakes, tasks, jobs and questions survive a restart, and due wakes ran 0.35 s after serving (Claude Code's "remind me in 2 minutes" died with its process). The experience is not there yet: 3 of the 11 horizon tests pass, and a reminder set from the CLI reaches no one.

The design. Every future thing is a wake with a purpose: a **reminder** (it tells the owner, no model call), a **check-in**, a **deferred task** (opened asleep, so it holds no budget all week), a **routine** (a fresh task a run, with its own budget and pre-approved tools), and a **watch** (a probe with \$0 of model until a condition holds); **projects** are dated milestones with a weekly review. All of it lands on one work board, bucketed Now, Today, This week, Later, Recurring, Waiting on the world and Projects, and the model can see, move and cancel what it promised.

Plan: [north star §1.6](#).

5. Memory: one store with lineage

The pitch. "It remembers every session I've had with it, and with my old agent, and when it tells me something from memory it links the exact message."

Today.

- **Recall is live** in front of the model on the owner's daemon, place-safe, with a judged rerank. On the memory exam it lifts a cheap model from 14.6% with no recall to 67.1%; handed the right notes the same model passes 99.5%, so what is left to win is retrieval.
- **History came along:** 21,779 episodes of the owner's earlier assistant history imported in 2 minutes 4 seconds, none rejected.
- **Not yet:** never measured head to head; off on a fresh install; questions about time and corrected values score 0% in every arm; recall's vector half misses its 250 ms deadline on the owner's daemon (theseus-zo1y fixes it).

Why Theseus can crush it. One store holds every message with its time, place and WAL position, and recall references sources instead of copying them. So a reply can **cite the message it remembers** (provable memory); a standing order can sit in every compiled prompt with a **manifest proving it was there** after any number of compactions; **one wrong fact can be traced everywhere it spread** and corrected; and **Claude Code's and Codex's history can come along** on day one. Claude Code's memory is a file per directory that cites nothing.

Plan: [north star §1.3](#): seven tests, among them the recall bench against Claude Code and Pi, compaction survival, and citations at least 90% right.

6. Personality

The pitch. "It knows who it is and who I am, in my terminal, in Discord and in a voice call, after every compaction and model switch, and it reads the moment."

Today. Behind: the whole persona is three fixed sentences. A probe with one 1 KB persona file passed every check: it knew its name, the owner's card and a standing preference in every new session, and answered a tired "quick one" in 102 tokens instead of 534. It cost 395 cached tokens and no extra call, and every compiled request names the persona and its digest, so **presence is provable from the manifest**, which Claude Code cannot match.

The design. An owner-shaped persona under Theseus's own name; standing preferences and orders in every prompt; the surface it is speaking on named to the model; a voice of its own; a personality exam in about ten families against Claude Code bare and Claude Code given the same persona (the fair arm), with blind pairs for the owner.

Plan: [north star §1.4](#).

7. Remote runtimes

The pitch. "The daemon runs on my server or in my AWS account, the CLI and TUI on my laptop, the index on another machine, and nothing local got slower."

The design. The same newline JSON-RPC as the Unix socket, inside TLS 1.3 on TCP, off unless `[rpc] listen` names it. mTLS with a private CA held as `op://` refs, each client a named device with a role (owner, observer, index), and a remote client its own surface with no host-path reads and a short list of local-only methods. A remote index follows a WAL feed over one persistent connection. At 30 ms of round trip a cold ask reaches its first event about 90 ms later, and recall keeps about 220 ms of its 250 ms deadline.

Status. Designed and decided (the owner took all twelve defaults); ten rows filed under theseus-pp7c, starting after the benchmark rerun.

Plan: [remote runtimes \(PDF\)](#).

8. Speculative activity

The pitch. "While I read its answer or step away, it guesses what I'll ask next and does that work in a throwaway copy. When I come back the diff and the passing tests are there: one key to use it, one to throw it away."

The design. The idea came from a collaborator and was shaped by the owner. A ladder from nearly free to bold: S0 keeps the next answer fast (the cache warm, recall pre-run); S1 scouts read-only in the sandbox; S2 does the whole likely next task in a speculation workspace with no network, no secrets and no way to touch the owner's files; S3 tries several futures; S4 runs heavy work on the AWS hands. It runs only when the machine is idle and pauses on any input, so FAST is never hurt; it spends from its own \$10 a day; every prediction is scored, so it learns where to guess and stops where it is wrong. Nothing reaches the owner's tree without **Use**.

Status. Designed and decided; thirteen rows filed under theseus-upf0, starting after the benchmark rerun. Its cheapest rung is already covered by the stopping point's cache, connection and recall fixes.

Plan: [speculation \(PDF\)](#).

| 9. Discord as a live surface

The pitch. "A lively chat with its thinking and tool lines, a calm task house in my guild, and it speaks first, in its own voice, when something happens."

Today. Tool lines stream into the chat; a typed DM reaches its first text within 58 to 88 ms of the model's first byte; voice calls take barge-in. Claude Code has no Discord surface at all. Not yet: thinking never reaches Discord, and Theseus speaks first only sometimes, in a system's words.

The design. The chat stays lively and on by default for the owner: thinking streams into each loop's message and folds to "thought for 4 s" when the answer starts, with four keys per place for anyone who wants it quieter. **Theseus speaks first** on 15 events (a question, a reminder, a finished or failed task, a routine's run, a watch met, what consolidation noticed, spend crossing a line, a morning note, and more), paced so the owner's turn comes first. **A task house** in the owner's guild: a board that never scrolls, a forum post per task under the task's own name, journal, ledger and health channels. A busy day stays far inside Discord's rate limits, and no turn gains a frame.

Plan: [north star §1.7](#).

Also in the plan

The killer-features investigation ranked more than thirty candidates. Beside the nine above, the strongest are already built and need only their proof: **"why did it do that?"** (every compiled request's manifest, with why each recalled note was there, checked against the digest of what was sent), and **keys that never reach the model** (one scrubber in front of the model, the log and every client, which knows the vault's actual values). Their rows are in the [north-star plan](#).

Sources outside the repo

The killer-features investigation, the scale and personality reports, and the design briefs behind sections 3, 7 and 8 are kept in the working investigations outside this repository. Every number above is also in a page linked from its section.